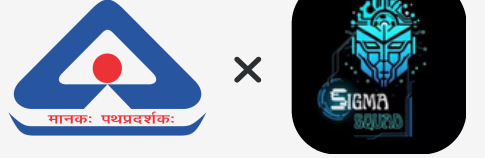


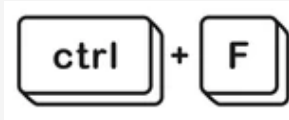
PROBLEM STATEMENT



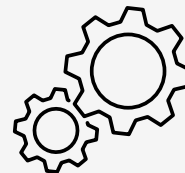
- **The Scale of the Problem:** Indian Micro and Small Enterprises (MSEs) are the backbone of the economy, but they lose critical weeks navigating labyrinthine regulatory documents.



- **The Data Monolith:** The core dataset (BIS SP 21) is a dense, 900+ page technical manual. It is filled with complex tables, OCR irregularities, and deeply nested sub-standards (e.g., Part 1 vs. Part 2).



- **The Flaw in Traditional Search:** Lack of Accessible Decision Support Tools
Existing systems rely on keyword-based or manual lookup, which:
 - Fails to capture semantic intent
 - Requires domain expertise

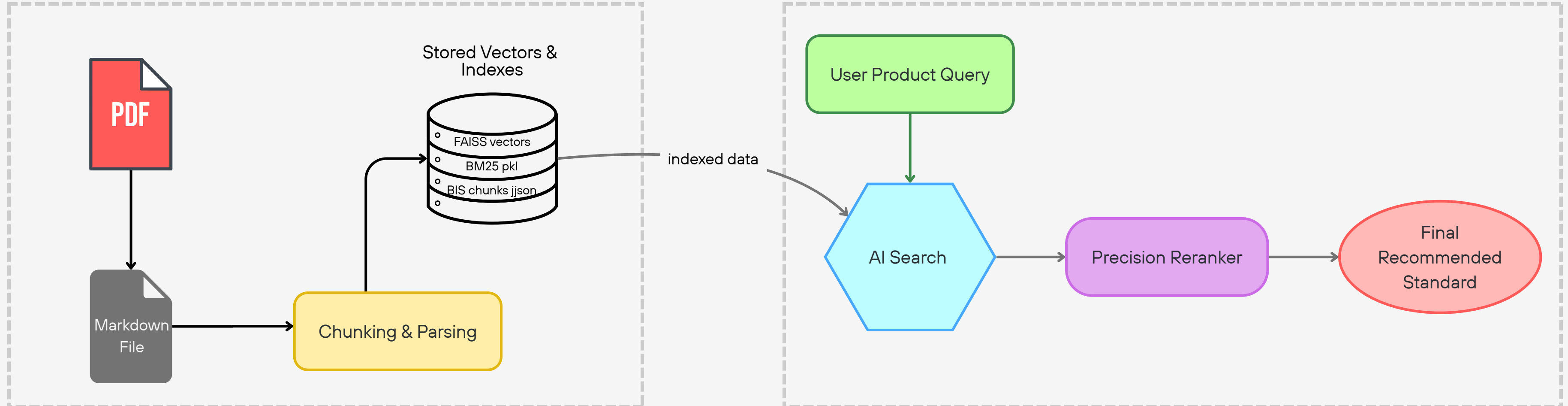


- **Need for Intelligent Automation:** There is a critical need for a system that can:
 - Understand natural language product descriptions
 - Map them to relevant BIS standards accurately and instantly
 - Provide explainable recommendations





SYSTEM ARCHITECTURE (High level view)



Phase 1: Intelligent Ingestion & Indexing

- Data Parsing: Processed complex SP 21 tables via MinerU.
- Smart Chunking: Custom Regex algorithm isolates individual sub-standards into self-contained JSON "Flashcards".
- Dual-Indexing: Pre-computed dense vectors (FAISS) and sparse word frequencies (BM25) for offline readiness.

Phase 2: Hybrid Retrieval (Casting a Wide Net)

- Semantic Search : Catches conceptual intent (e.g., "water mains" → "concrete pipes").
- Keyword Search : Enforces strict compliance matching.
- The Funnel: Combines the unique top candidates (k=15) from both engines.

CHUNKING & RETRIEVAL STRATEGY



Chunking :

```
{
  "standard_id": "IS 269: 1989",
  "content": "IS 269: 1989 33 grade ordinary Portland cement (fourth revision) 1.10"
},
{
  "standard_id": "IS 2185 (Part 2): 1983",
  "content": "IS 2185 (Part 2): 1983 Concrete masonry units Part 2 Hol
```

bis_chunks.json : extracted after processing

Regular Expression:

```
r"(IS\s+\d+(?:\s*\(\s*Part\s*\d+\s*\))?\s*:\s*\d{4})"
```

Standard AI chunkers slice text blindly by character limit. Technical documents like BIS SP 21 are highly structured. If you use a blind chunker, it might slice the document right in the middle of a section.

- **Chunk A** gets the title: "IS 2185 (Part 2): 1983 Specification for Concrete Masonry Units"
- **Chunk B** gets the details: "These are lightweight hollow blocks used for..."

Retrieval :

1. Query Pre-Processing

- The query is lowercased and split into individual tokens
- The query is passed through your [all-MiniLM-L6-v2](#) embedder model. This converts the English sentence into a high-dimensional vector array

2. Dual-Branch Retrieval

Branch A: Sparse Search (BM25) :

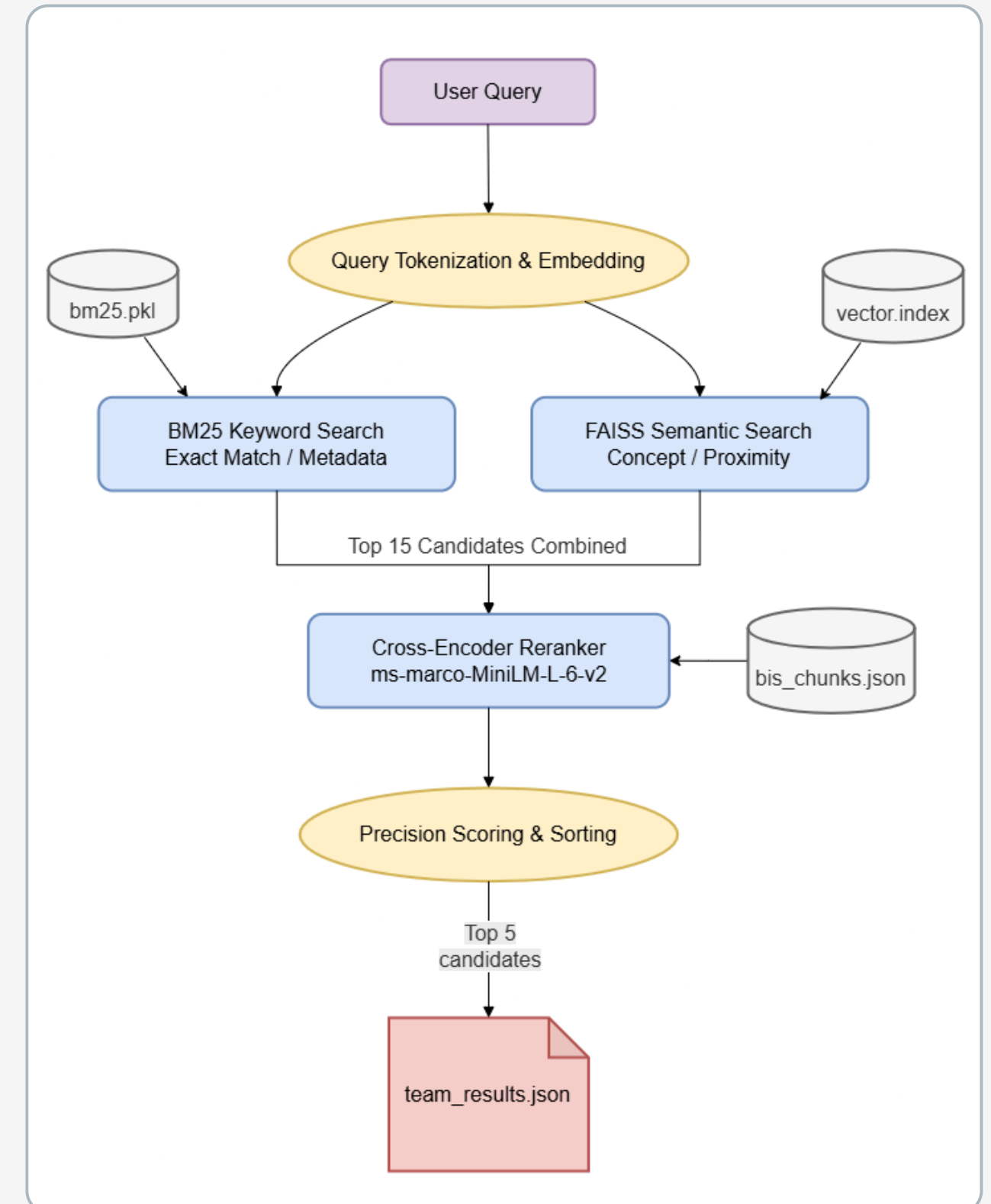
- How it works: It scans your index for exact word frequencies.
- Why it matters: If a user specifically asks for "33 Grade" or "IS 455", [BM25](#) ensures those exact alphanumeric characters are heavily weighted and not ignored.

Branch B: Dense Vector Search (FAISS)

- How it works: It takes your query vector and looks for the "nearest neighbors" in your FAISS database.
- Why it matters: It understands concepts. If the user searches for "water mains," FAISS knows mathematically that "concrete pipes" is a highly related concept, even if the exact words don't match.

3. Precision Reranking

The user's original query is taken and physically pair it with the text of candidate #1. You feed that pair [query, chunk_text] into your [ms-marco Cross-Encoder](#).



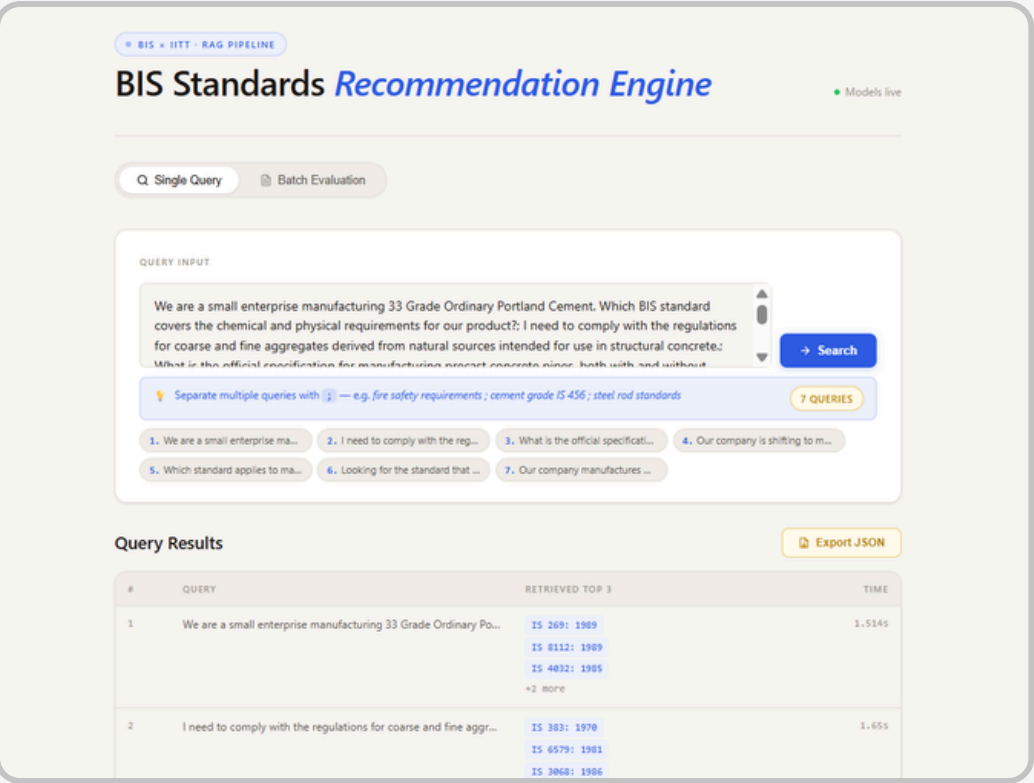
RAG Pipeline

DEMO HIGHLIGHTS



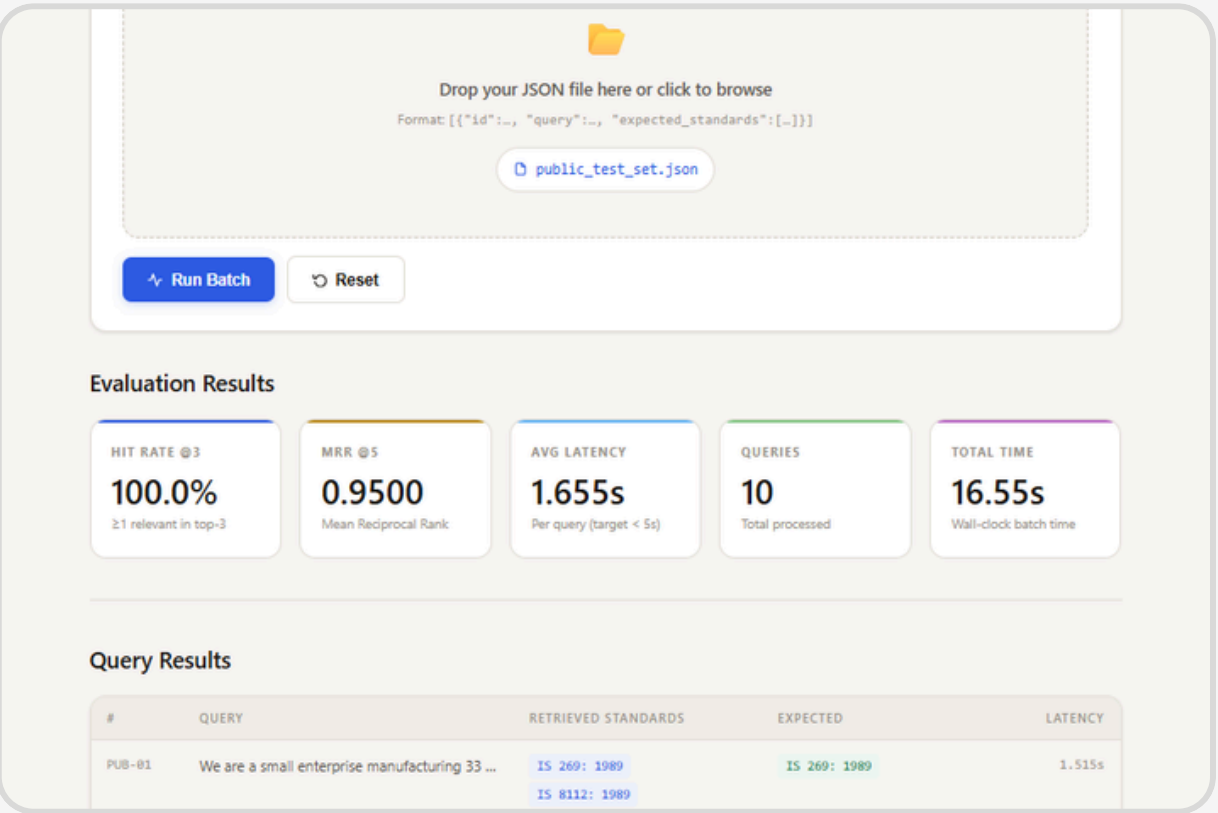
Built a robust, high-performance RAG pipeline and demonstrated it through a fully functional FastAPI server with a modern web interface.

1. Manual Input Query Mode



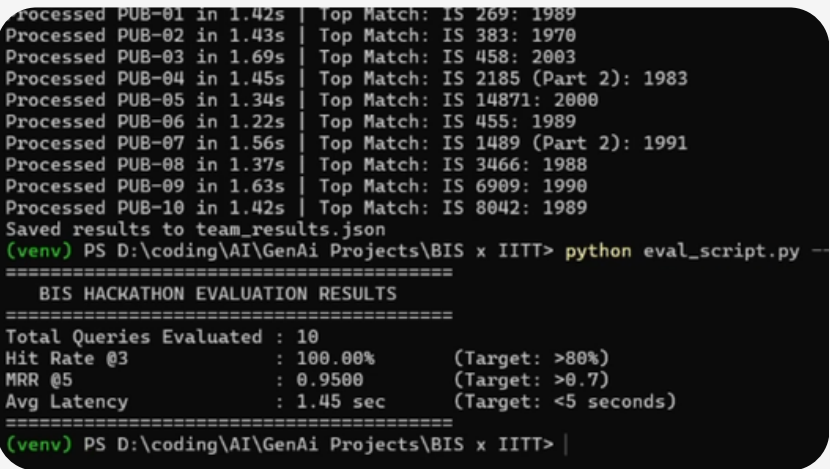
- Entered a product description manually into the interface
- Supports **multiple queries** separated by ";" for batch processing
- The pipeline instantly retrieved the **Top 5** most relevant BIS standards
- The UI displayed:
 - Ranked results (best match highlighted at the top)
 - Confidence-based ranking order
 - Total latency
- Results can be exported** in structured JSON format for further use

2. Batch Processing Mode



- Uploaded a **structured JSON** test file containing multiple queries
- With a single click, the system processed all queries within **seconds**
- The interface dynamically displayed:
 - Hit Rate, MRR, and Average Latency
 - Complete list of queries with:
 - Retrieved standards
 - Individual response times

Additional Demonstration



showed that the entire pipeline is accessible via **Command Line Interface**

- ✓ Run queries
- ✓ Evaluate results

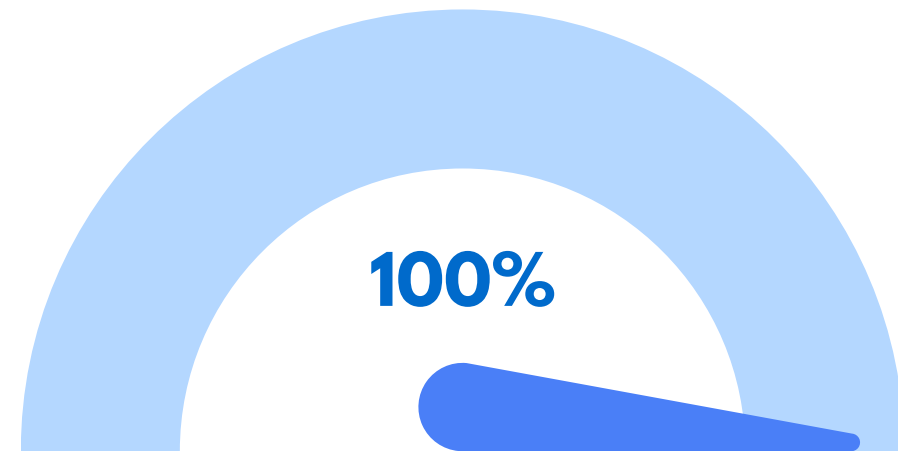
System Capabilities Demonstrated

- Real-time inference with high accuracy and speed
- Seamless handling of both single and bulk queries
- Fully offline execution
- Clean and intuitive UI for non-technical users

EVALUATION RESULTS



Evaluation is performed on the public test set given



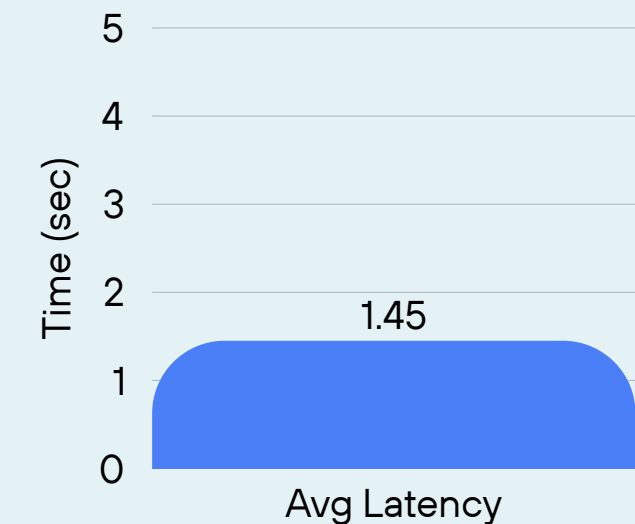
Hit Rate @3

- Every query retrieved at least one correct BIS standard in the top-3
- Achieved >80%, indicating most queries returned at least one correct BIS standard in top-3 results

0.95

MRR @5 (Mean Reciprocal Rank)

- Achieved ~0.7+, showing correct standards are ranked highly (often in top positions)
- Correct standards consistently ranked at top positions



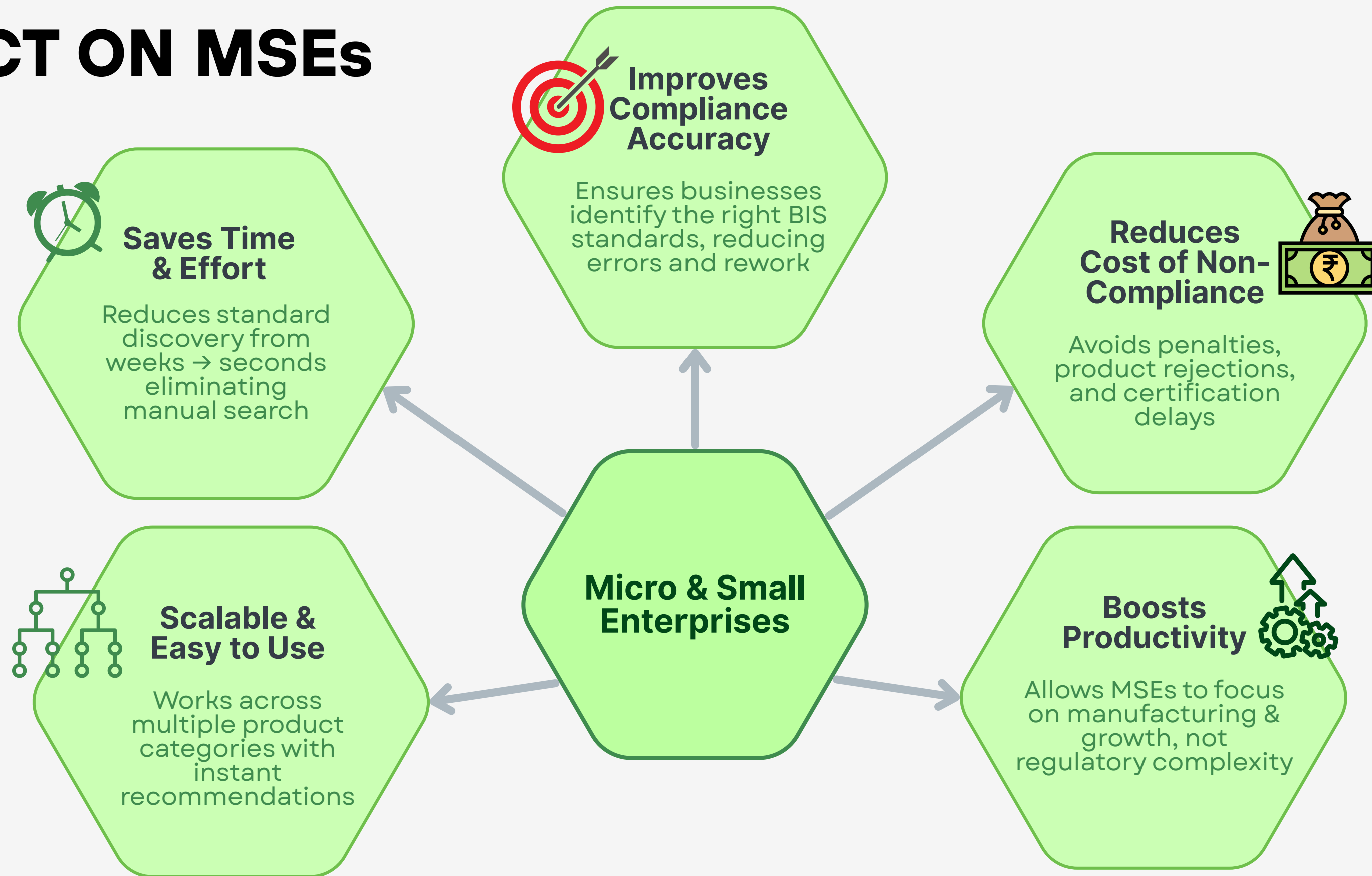
Average Latency

- 1.52 seconds Fast, near real-time response per query
- < 1–2 seconds per query, enabling real-time recommendations

No Hallucinations

Maintained high factual accuracy with valid BIS standard outputs by only accessing the information from the extracted data, thus not generating a new one.

IMPACT ON MSEs



“Empowering MSEs to achieve faster, smarter, and more reliable compliance.”

TEAM & ACKNOWLEDGEMENTS



References :

- **Dense Embedding Model :** [sentence-transformers/all-MiniLM-L6-v2](#)
- **Precision Reranking Model:** [cross-encoder/ms-marco-MiniLM-L-6-v2](#)
- **Sparse Keyword Indexing (BM25):** [rank-bm25](#) Implementation.
- **Vector Similarity Search:** [FAISS \(Meta AI Research\)](#).
- **Document Ingestion (PDF to Markdown):** [MinerU](#) (OpenDataLab)

Team Name: **Night Owl**

Team Member: **Harshit Kushwaha (3rd B.Tech Computer Engg.)**

College Name: **Sardar Patel Institute of Technology (SPIT), Mumbai**

Thank You!

 **Repo** ↗

 **Demo** ↗